

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 3 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	M.Rishitha	Reg. No.:	192424335
Section / Batch:	2024 AIDS	Date:	20/8/2026

Code of Conduct

I M.Rishitha[192424335] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.Rishitha

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Annexure A – Design Task

Task 1 – N-gram Based Text Prediction

Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.

For an incomplete sentence such as:

"Artificial intelligence is"

the system should identify and display the top-5 most probable next words.

The program should allow the user to select N = 1, 2, or 3, display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.

Program Output (Task 1) — executed successfully:

```
Terminal - task1_ngram.py

Total sentences after segmentation: 10
Sample tokenized sentence: ['<s>', 'artificial', 'intelligence', 'is', 'transforming', 'the',
'world', 'of', 'technology', '</s>']
-----
Unigram frequency (top 10):
[('<s>', 10), ('</s>', 10), ('artificial', 9), ('intelligence', 9), ('is', 7), ('of', 5), ('the',
3), ('learning', 3), ('a', 3), ('and', 2)]
-----
Bigram counts for context ('artificial',): Counter({'intelligence': 9})
Trigram counts for context ('intelligence','is'): Counter({'transforming': 1, 'used': 1, 'helping':
1, 'changing': 1})
-----
N=1, context=()
<s>          P=0.097
</s>         P=0.097
artificial   P=0.087
intelligence P=0.087
is           P=0.068

N=2, context=('is',)
a            P=0.429
transforming P=0.143
used         P=0.143
helping      P=0.143
changing     P=0.143

N=3, context=('intelligence', 'is')
transforming P=0.250
used         P=0.250
helping      P=0.250
changing     P=0.250

Unseen trigram context test: ('quantum', 'computer') -> predictions: NONE (zero probability problem)
-----
Prediction accuracy test (trigram model, top-5):
Context: 'artificial intelligence is' | Actual next: 'transforming' | Top-5: ['transforming',
'used', 'helping', 'changing'] | HIT
Context: 'machine learning is' | Actual next: 'a' | Top-5: ['a'] | HIT
Context: 'deep learning is' | Actual next: 'a' | Top-5: ['a'] | HIT
Context: 'natural language processing is' | Actual next: 'a' | Top-5: ['a'] | HIT
Context: 'the future of' | Actual next: 'artificial' | Top-5: ['artificial'] | HIT

Top-5 Prediction Accuracy: 100.00%
-----
Limitation: The unsmoothed trigram/bigram model assigns ZERO probability
to any n-gram not seen in training (e.g., 'the quantum computer' above),
making it brittle for real-world, open-vocabulary text prediction.
```

Task 2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:

"Machine learning can"

When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights.

The system should generate the top-5 next-word predictions using:

- Unsmoothed N-gram model
- Backoff model
- Deleted Interpolation model

Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

Program Output (Task 2) — executed successfully:

```
Terminal - task2_backoff.py

Test sentence: 'machine learning can'
=====
1) UNSMOOTHED N-GRAM MODEL (trigram only):
[('predict', 0.14285714285714285), ('improve', 0.14285714285714285), ('automate',
0.14285714285714285), ('recognize', 0.14285714285714285), ('help', 0.14285714285714285)]

2) BACKOFF MODEL (used: trigram):
predict      P=0.143
improve      P=0.143
automate     P=0.143
recognize    P=0.143
help         P=0.143

3) DELETED INTERPOLATION MODEL (lambda1=0.1, lambda2=0.3, lambda3=0.6):
predict      P=0.124
improve      P=0.124
recognize    P=0.124
help         P=0.124
generate     P=0.124
=====

Unseen-context comparison for: 'quantum computers can'
Unsmoothed : ZERO PROBABILITY (fails)
Backoff     : (bigram (backoff)) [('predict', 0.125), ('improve', 0.125), ('automate', 0.125),
('recognize', 0.125), ('transform', 0.125)]
Interpolation: [('predict', 0.03867647058823529), ('improve', 0.03867647058823529), ('recognize',
0.03867647058823529), ('help', 0.03867647058823529), ('generate', 0.03867647058823529)]

=====
COMPARISON SUMMARY
=====
Model      Zero-Prob Cases  Coverage
Unsmoothed  High             Low
Backoff     Low              High
Interpolation None             Full

Backoff and interpolation are useful because real corpora are sparse:
many valid trigrams never appear in training. Backoff falls back to
lower-order n-grams when higher-order counts are zero, while deleted
interpolation blends all orders together, giving smoother, more robust
probability estimates and avoiding the zero-probability problem entirely.
```

Task 3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.

For example, compare sentences such as:

"The cat sits on the mat."

and

"The quantum processor redesigned the..."

The system should evaluate how predictable the next word is based on the trained language model.

The program should also compare entropy values obtained using:

- Unigram model
- Bigram model
- Trigram model
- Smoothed model

Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.

Program Output (Task 3) — executed successfully:

```
Terminal - task3_entropy.py

Training sentences: 8 | Vocabulary size: 26
-----
ENTROPY COMPARISON ACROSS MODELS

Sentence                                Unigram  Bigram   Trigram  Smoothed(tri)
-----
The cat sits on the mat.                 3.859    1.116    5.543    3.832
The quantum processor redesigned the alg... 20.009   28.474   33.219   4.756
-----

INTERPRETATION
-----
'The cat sits on the mat.'
  Smoothed trigram entropy = 3.832 bits -> LOW uncertainty (closer to training data)

'The quantum processor redesigned the algorithm.'
  Smoothed trigram entropy = 4.756 bits -> HIGH uncertainty (unfamiliar structure/vocabulary)

Entropy quantifies how 'surprised' the language model is by the test
sentence: lower entropy means the model assigns higher probability to
the observed words (more predictable, in-domain text), while higher
entropy indicates unfamiliar vocabulary/structure and lower model
reliability. Smoothing reduces artificially inflated entropy caused by
zero-probability n-grams in an unsmoothed model.
```

Task 4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.

The system should implement three approaches:

1. Rule-Based POS Tagging

Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.

2. Stochastic POS Tagging

Construct a statistical POS tagger using:

- Word/tag frequencies
- Tag transition probabilities
- Probabilities obtained from a training corpus

3. Transformation-Based Tagging

Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.

For example:

"I book a ticket."

and

"I read a book."

The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches

and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

Program Output (Task 4) — executed successfully:

```
Terminal - task4_postag.py

COMPARATIVE POS TAGGING RESULTS
=====

Sentence: "I book a ticket."
Word      Rule-Based   Stochastic   Transform-Based
-----
I          PRP          PRP          PRP
book       NN          VBP          VBP
a          DT          DT           DT
ticket     NN          NN           NN
.          .           .            .

Sentence: "I read a book."
Word      Rule-Based   Stochastic   Transform-Based
-----
I          PRP          PRP          PRP
read       NN          VBP          VBP
a          DT          DT           DT
book       NN          NN           NN
.          .           .            .

=====
KEY OBSERVATION: context-dependent tagging of the word 'book'
=====
In "I book a ticket." -> 'book' should be a VERB (VBP)
In "I read a book."   -> 'book' should be a NOUN (NN)

EVALUATION (tagging accuracy vs gold-standard tags)
-----
Rule-Based           Accuracy: 80.00%
Stochastic           Accuracy: 100.00%
Transformation-Based Accuracy: 100.00%

=====
DISCUSSION
=====
Rule-Based: Fast and interpretable but relies on hand-written rules and a fixed lexicon; struggles with words unseen in the rule set.
Stochastic: Learns word/tag and transition probabilities from a training corpus, so it generalizes better and captures context (e.g., pronoun followed by a determiner implies a verb), but needs sufficient training data to estimate reliable probabilities.
Transformation-Based: Starts from a simple baseline tag and applies learned correction rules, combining the interpretability of rules with data-driven refinement; accuracy depends on the quality and coverage of the transformation rules.
```

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis

Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation
--------------------------------	-----	---	--------------------------------	--------------------------------	-------------------------------

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1	5	5	5	5	5	25	95
2	5	5	5	5	4	24	
3	5	5	5	4	4	23	
4	5	5	4	4	4	22	

Faculty In-charge	Course Coordinator	Program Director
T.KUMARAGURUBARAN		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____